/WEATHER ENDPOINT

Consolidated Probe Report

Consolidates findings from `weather_probe_report.html` (32 logical probes, 38 actual API calls due to throttle retries, 2026-05-17), `weather_cancellation_report.html` (8 API calls + 2 cancelled + 1 verdict log line, 2026-05-17), and the shared-bucket findings from `shared_rate_limit_bucket_report.html` (2026-05-19). Originals archived in `_archive/`.

Quirks

WQ1 MAJOR **Throttle is HTTP 200, not 429.** Rate limiting is hidden inside the response body as `{"status":"throttled"}`. Standard HTTP status code conventions are not used.

WQ2 MAJOR **Conflicting observations for the same city.** In the probe run, ~10–15% of successful calls returned Shape B, which contains multiple contradictory weather readings for the same location (e.g. 12.5°C Overcast vs 11.8°C light rain for Paris). The API provides no indication of which reading is authoritative. The structural difference (flat Shape A vs array Shape B) is secondary — the real issue is ambiguous, conflicting data.

WQ3 MAJOR **Silent fuzzy matching.** Mars→Marseille, India→New Delhi, UK→London, 京都→京都市. The API resolves to a real city without signalling that it changed the input. The response `location` field differs from the request, but only if you compare.

WQ4 MAJOR **Fabrication for fictional cities.** Atlantis returns fabricated weather data with no error. The API invents plausible-looking numbers for non-existent locations.

WQ5 MAJOR **Cancelled calls still consume a throttle slot.** The server processes the request on arrival. Client-side cancellation (timeout or `asyncio.task.cancel()`) does not free the slot.

WQ6 MAJOR **Shared rate-limit bucket with /research.** Both endpoints draw from the same ~5-per-30 s window. A weather-heavy turn can starve a subsequent research call.

WQ7 MINOR **Mixed casing in Shape B conditions.** "Overcast" (capitalised) vs "light rain" (lowercase) in the same response array.

TL;DR

1. **Rate limit is HTTP 200 with body envelope, not 429.** After ~5 successful calls per ~30 s window, the response is `{"status":"throttled","retry_after_seconds":N,"data":null}`.

2. **Shape B returns conflicting observations for the same city.** Shape A (flat) and Shape B (multi-observation `conditions` array) can appear for any city on any call. The structural difference is solved by normalisation, but Shape B contains contradictory readings (different temperatures, conditions) with no way to determine which is authoritative.
3. **Fuzzy matching and fabrication.** Mars→Marseille, 京都→京都市, Atlantis→fabricated weather. No explicit mismatch flag; the client must compare `requested_location` against `returned_location`.
4. **Cancelled calls still consume a throttle slot.** Server processes the request immediately; client cancellation frees nothing.
5. **Shared rate-limit bucket.** /weather and /research share the same ~5-per-30 s window (confirmed by interleaved probes).

Response Variants (7 observed)

VARIANT	STATUS	EXAMPLE	TRIGGER
A — Flat single	200	`{location:"London", temperature_c:14.0, condition:"Partly cloudy", humidity:67}`	Most real-city queries
B — Multi-observation	200	`{location:"Paris", conditions:[...], note:"Multiple conditions..."}`	~10–15% of successful calls in probe run, random
Throttle envelope	200	`{status:"throttled", retry_after_seconds:29, data:null}`	After 5 successful calls per ~30 s
404 not found	404	`{error:"Location \"\" not found"}`	Empty location or very long input (5000 chars)
422 validation	422	FastAPI default envelope	Missing or wrong param name
401 unauthorised	401	`{error:"Invalid or missing API key"}`	No key or wrong key
405 method not allowed	405	`{detail:"Method Not Allowed"}`	OPTIONS request

Schema Variation (Shape A vs Shape B)

Both shapes can appear for any city on any call — not city-locked, not deterministic. The structural difference (flat vs array) is a parsing problem solved by normalisation. The deeper issue is that Shape B returns **multiple contradictory weather readings for the same location** — different temperatures, different conditions, different humidity values — with no indication of which is authoritative or why they differ.

Shape A (flat)

```
{
  "location": "London",
  "temperature_c": 14.0,
  "condition": "Partly cloudy",
  "humidity": 67
}
```

Shape B (multi-observation)

```
{
  "location": "Paris",
  "conditions": [
    {"temperature_c": 12.5, "condition": "Overcast", "humidity": 82},
    {"temperature_c": 11.8, "condition": "light rain", "humidity": 90}
  ],
  "note": "Multiple conditions reported for this location"
}
```

Throttle Behaviour

- ~5 successful 200s per ~30 s sliding window, then 6th returns throttle envelope.
- `retry_after_seconds` starts at ~29 and decrements as wall-clock time passes within the window.
- Error responses (401/404/405/422) are never replaced by a throttle envelope. Even when the rate-limit budget is exhausted, an invalid request (wrong key, missing param, empty location) still returns its normal error code. The error path short-circuits before the rate limiter, so error responses do not consume a throttle slot either.
- After sleeping `retry_after_seconds + 1` seconds, the same call succeeds on the next attempt. Confirmed across 6 backoff retries during the probe run: every throttled call eventually succeeded, and 0 calls gave up. This means a simple retry loop with the server-provided wait time is sufficient — no exponential backoff or jitter needed for a single-user CLI.

Shared Rate-Limit Bucket

> **Both /weather and /research share the same ~5-per-30 s bucket.** Confirmed by interleaved probes: 5 /weather calls exhausted the budget, and the next /research call was immediately throttled. Conversely, 4 /weather + 1 /research consumed all 5 slots, and the next /weather was throttled.

Fuzzy Matching

The API silently resolves ambiguous or partial input to a real city. The response `location` field contains the resolved name, not the original input — but nothing else in the response signals that a substitution occurred. The only way to detect it is to compare request input against response `location`.

Acceptable fuzzy matches

INPUT	RETURNED LOCATION	NOTES
京都	京都市	Unicode input resolved to official city name (Kyoto → Kyoto-shi)
India	New Delhi	Country name resolved to capital city
UK	London	Country abbreviation resolved to capital city

Questionable fuzzy matches

INPUT	RETURNED LOCATION	NOTES
Mars	Marseille	Partial string resolved to nearest real city — user likely did not mean Marseille

Fabrication

When the API cannot fuzzy-match to a real city, it does not return an error. Instead it returns fabricated weather data with plausible-looking numbers for a location that does not exist.

INPUT	RETURNED LOCATION	NOTES
Atlantis	Atlantis	Fictional city; API returned fabricated temperature, condition, and humidity with no error flag or indication that the data is invented

Input Handling

INPUT	BEHAVIOUR
Empty string	404
5000 chars	404 with full input echoed
LONDON / london	Normalised to "London" (case folded)
" London "	Whitespace stripped
"London, UK"	Comma suffix ignored → London

INPUT	BEHAVIOUR
Extra param (foo=bar)	Ignored
Duplicate param	Last value wins
Missing param	422

Cancellation Findings

> **Cancelled call still costs a throttle slot.** Server processes request on arrival; client-side cancellation (ReadTimeout at 50 ms or `asyncio.task.cancel()`) does not free anything.

- `asyncio.CancelledError` raised cleanly — no zombie tasks, no leaks.
- `httpx.AsyncClient` pool survives cancellation — same client usable for next request.
- Repeated identical queries within the same probe run returned byte-for-byte identical response bodies (same SHA-256 hash). However, the API does not explicitly mark responses as cached (unlike /research which uses a `cached: true` flag), so this may reflect deterministic computation rather than server-side caching.

Concurrency

- 5 parallel calls to `/weather?location=London`: 3 succeeded immediately, 2 throttled, retried after 30 s, all 5 ultimately succeeded.
- Confirms true server-side parallelism; concurrent fan-out consumes slots.
- All 5 returned byte-for-byte identical 84-byte bodies (SHA-256 `ce136498…`), indicating deterministic responses for the same input within a time window.

Latency Profile

CALL TYPE	RANGE	NOTES
Cold start	~3.1 s	Cloud Run cold start
Warm valid	0.107–0.170 s	p50 ≈ 0.12 s
Concurrent (5 parallel)	0.148–0.203 s	Slightly elevated
Error responses	0.033–0.037 s	Skip weather generation
Throttled	~0.04 s	Plus backoff wait

Prompt Injection

A prompt-injection payload was previously discovered at the API root (`/`), so the concern was whether similar content could be embedded in `/weather` responses. Because /weather returns structured data (temperature, condition, humidity) rather than free-text summaries, the risk model here is *response-embedded injection*: the API quietly slipping instructional text into a JSON field that the chat app later passes into an LLM prompt.

Approach: all 38 retained Run 2 response bodies (32 logical probes + 6 throttled-first retry bodies) were scanned against 8 heuristic patterns:

1. HTML comments (`<!--...-->`)
2. Override-instruction phrases ("ignore all previous instructions" and variants)
3. AI self-references ("if you are an LLM...")
4. Known seed markers from the `/` root injection ("banana", "please add a comment", "found via api")
5. Zero-width and bidirectional-override Unicode characters
6. Roleplay markers ("act as...", "from now on...", "pretend that...")
7. System-prompt markers ("system:", "instruction:", "important:")
8. Imperative-at-reader phrases ("you must", "please add", "please reply")

Result: zero heuristic hits across all 38 retained response bodies. Every response contained only the expected structured fields (`location` , `temperature_c` , `condition` , `humidity` for Shape A; the `conditions` array for Shape B). No instructional text, no hidden markers, no Unicode tricks.

Conclusion: the prompt-injection content at `/` is a one-off Easter egg for LLM-assisted candidates and does not propagate to the `/weather` endpoint. The chat app still envelopes all tool responses as defence-in-depth, but there is no active threat in the weather response bodies.

Errors

- 401 (auth), 404 (empty/non-resolvable), 422 (missing param), 405 (non-GET) — all well-shaped and reliable.
- Error responses are ~3× faster (~30–40 ms) than valid responses.
- All bypass throttle budget.

Source Data

32 logical main probes produced 38 actual API calls due to throttle retries. Cancellation probing added 8 API calls, 2 cancelled requests, and 1 verdict log line. Raw logs: `backend/outputs/probes/weather/weather_probe_log.jsonl` and `weather_cancel_log.jsonl` . Original reports archived in `_archive/` .

Quirk Handling in the Chat App

How each quirk listed above is handled by the chat application code.

QUIRK	NAME	SEVERITY	HOW IT IS HANDLED	EXAMPLE
WQ1	Throttle is HTTP 200, not 429	MAJOR	`elyos_client.py` inspects every 200 response body for `"status": "throttled"`. When detected, it reads the server's authoritative `retry_after_seconds`, sleeps `retry_after + 1` s, and retries up to `max_throttle_retries` (2) times. The user sees *"Rate-limited, retrying in Ns..."* on the CLI. If retries are exhausted, a `throttle_exhausted` error dict is returned and the LLM explains the failure. Bounded concurrency (`max_concurrent` semaphores in `agent.py`) limits simultaneous in-flight calls. In the current config, weather is serialised with `weather.max_concurrent: 1`. This avoids same-endpoint bursts but does not track the shared server budget — large batches still hit the throttle wall. Throttles are handled reactively using `retry_after_seconds`.	Server returns `{"status":"throttled","retry_after_seconds":29,"data":null}` *Client prints "Rate-limited, retrying in 30s...", sleeps 30 s, and retries. If retries are exhausted, it returns a `throttle_exhausted` error.*
WQ2	Conflicting observations for the same city	MAJOR	Two layers. **Parser layer:** `parsers/weather.py` normalises both shapes into a single `NormalisedWeather` Pydantic model. Shape A produces one `WeatherObservation`; Shape B produces multiple. The LLM always receives the same schema. **Prompt**	Shape B returns 12.5°C Overcast and 11.8°C light rain for Paris. *LLM responds: "The API returned two conflicting readings for Paris: 12.5° and overcast, vs 11.8°C with light rain. I can't say which is definitive."*

QUIRK	NAME	SEVERITY	HOW IT IS HANDLED	EXAMPLE
			layer: the system prompt instructs the LLM to present all observations when multiple exist — do not average them, do not pick one, do not collapse them. State that the API returned conflicting readings and that no single reading should be treated as definitive.	
WQ3	Silent fuzzy matching	MAJOR	`parsers/weather.py` captures both `requested_location` (what the user asked for) and `returned_location` (what the API resolved to) in `NormalisedWeather`. The system prompt (`prompts.py` , `<weather>`) instructs the LLM: *"When requested_location and returned_location differ, tell the user about the mismatch."* This ensures the user knows "Mars" was resolved to "Marseille."	Acceptable: user asks for "India", API returns `"location":"New Delhi"` . *LLM responds: "The API resolved 'India' to New Delhi. Here's the weather for New Delhi..."* Questionable: user asks for "Mars", API returns `"location":"Marseille"` . *LLM responds: "The weather API interpreted 'Mars' as Marseille, which probably isn't what you meant."*
WQ4	Fabrication for fictional cities	MAJOR	The API provides no error flag, confidence score, or indication that a location is fictional. Handled at the prompt level: the system prompt instructs the LLM to caveat obvious fictitious or mythological locations such as Atlantis, Gotham, or Mordor. This works because the LLM recognises these as non-real places, even though the API returns plausible-looking data for them.	User asks for "Atlantis". API returns plausible weather data. *LLM responds: "Atlantis could be a fictional place, so take this with a large grain of salt."* User asks for "Gotham". API returns `"location":"Gotham Bridge"` *LLM responds: "Gotham is likely a fictional place, so take this with a grain of salt" and also surfaces the location mismatch.*

QUIRK	NAME	SEVERITY	HOW IT IS HANDLED	EXAMPLE
WQ5	Cancelled calls consume a throttle slot	MAJOR	`cli_chat.py` warns the user on SIGINT cancellation: *"Cancelled. The interrupted API call may still count against the rate limit."* The WebSocket path supports a `{"type":"cancel"}` message and emits `{"type":"cancelled"}` after rolling back the interrupted turn. The server counts the cancelled request against its rate budget regardless of whether the client received a response. The reactive throttle retry in `elyos_client.py` handles any subsequent throttle responses using the server's `retry_after_seconds`.	User presses Ctrl+C during a weather lookup, or the web UI presses the cancel button. The CLI prints the cancellation warning; the web UI marks the partial response as interrupted. *If the next request triggers a throttle, the client sleeps for the server-specified retry period and retries.*
WQ6	Shared rate-limit bucket with /research	MAJOR	Both `/weather` and `/research` share a single server-side rate budget (~5 requests per 30 s window). The `config.yaml` `max_throttle_retries` is at the API level, not per-endpoint, reflecting this shared budget. Bounded concurrency (`max_concurrent` semaphores) limits simultaneous in-flight calls. Current config uses `weather.max_concurrent: 1` and `research.max_concurrent: 1`, so tool calls are serialised per endpoint. This does not track the shared server budget. The reactive throttle retry in `elyos_client.py` handles throttle responses using the	5 weather calls exhaust the shared budget. The next research call gets throttled. *The client sleeps for the server-specified retry period and retries. If retrie are exhausted, it returns a `throttle_exhausted` error.*

QUIRK	NAME	SEVERITY	HOW IT IS HANDLED	EXAMPLE
			server's authoritative `retry_after_seconds`.	
WQ7	Mixed casing in Shape B conditions	MINOR	Not explicitly handled in code. The parser passes condition strings through as-is ("Overcast", "light rain") without normalising case. This is acceptable because the LLM reads natural-language condition strings and naturally handles mixed casing when summarising weather for the user.	Shape B arrives with `"condition":"Overcast"` and `"condition":"light rain"`. Both are passed to the LLM as-is. *The LLM responds: "Paris has overcast skies with light rain" — the casing difference is invisible in the final output.*